Slim-Mamba: An Efficient and Reconfigurable Mamba-Based Accelerator for Massive MIMO Indoor Localization

Shengjie Chen†, Chenxi Zhang†, Sijia Cheng, Ove Edfors, Liang Liu, and Ilayda Yaman

***Abstract*—Massive MIMO radio measurements provide rich spatial information and delay-domain information obtained through channel impulse response extraction, but their high-dimensional channel features lead to considerable computational and memory requirements, making real-time neural network inference challenging on edge devices with limited resources. This paper presents Slim-Mamba, an efficient and reconfigurable Mamba-style accelerator for Massive MIMO indoor localization. Mamba is a recent state-space sequence model, making it a promising fit for radio localization because its recurrent state updates can exploit previous measurements when estimating the current position in continuous indoor movement. We further propose a system-level solution using real-world Massive MIMO measurements from LuViRA dataset [2], algorithm optimization, and hardware acceleration for indoor localization. At the algorithm level, Slim-Mamba simplifies the original Mamba-style selective state-space block into a lightweight gated recurrent update, preserving temporal state modelling and output gating while reducing the complexity of the selective-scan computation. The deployed model contains 816k trainable parameters and a model size of 3.40 MB while maintaining high localization accuracy. At the hardware level, the accelerator supports reconfigurable architecture with a highly parallel MAC array optimized for matrix-vector operations and fine-grained pipelining, improving resource utilization and reducing inference latency. Implemented on a Zynq UltraScale+ FPGA, the accelerator achieves a mean localization error of 0.14 m, closely matching the floating-point baseline, and reaches 471 positions/s at 100 MHz.**

***Index Terms*—Massive MIMO, Indoor localization, Mamba, FPGA accelerator, quantization, reconfigurable architecture.**

I. Introduction

INDOOR localization is becoming an important function in future wireless systems, where communication, sensing, and positioning are expected to be more tightly integrated. In the context of 6G, localization has been identified as both a key service and an enabling technology for location-aware applications such as robotic navigation, emergency healthcare, smart environments, and intelligent transportation [3]. These applications impose diverse accuracy requirements, ranging from meter-level positioning for general indoor tracking to sub-meter [4] and decimeter-level [5] accuracy for emerging applications requiring more precise spatial awareness. However, accurate indoor positioning remains challenging because indoor radio propagation is affected by multipath effects, noise, and the surrounding radio environment [6].

Massive multiple-input multiple-output (MIMO) systems provide a promising measurement platform for radio-based indoor localization. The LuViRA dataset used in this paper provides synchronized vision, radio, audio, and motion-capture measurements for indoor localization research [2], [7]. In its radio modality, the channel response is measured using the Lund University Massive MIMO (LuMaMi) testbed [8]. Since radio-based localization itself is a regression task (from channel state information to user equipment positions), data-driven algorithms such as machine learning(ML) can also be utilized [9].

Previous works have explored ML-based methods for Massive MIMO indoor localization. Early studies adopted fully connected neural network (FCNN)-based approaches using spatial covariance matrices and truncated channel impulse responses (CIRs) as fingerprints, achieving centimetre-level positioning accuracy [9]. However, these methods rely on large fully connected layers, resulting in model sizes ranging from hundreds of megabytes to more than 1 GB. The extension in [10] further confirms that network size and computational cost remain major challenges when applying ML-based localization to Massive MIMO systems, potentially hindering training, fine-tuning, and hardware deployment. To reduce this burden, CNN-based methods exploit the structured nature of channel state information (CSI) fingerprints and use convolutional layers to extract local spatial features for position estimation [11]. The corresponding CNN accelerator was later incorporated into a broader ASIP platform [12]. Nevertheless, CNNs still introduce considerable computational cost and are mainly limited to extracting local features. More recently, attention-based methods are used in the field of localization. Self-attention can capture correlations between beam-domain channel responses, where pairwise similarity patterns provide valuable spatial information for user localization [**?**]. However, the quadratic complexity of self-attention, together with its associated memory traffic remains a challenge for efficient hardware implementation [13].

From the examples above, efficient deployment which reduces model complexity, computation, and memory movement while preserving accuracy still remains a critical challenge for learning-based Massive MIMO localization. Mamba [14], based on a state-space model(SSM) that replaces self-attention and conventional MLP blocks with selective state-space processing and gating, provides a promising direction for this purpose. Mamba performs recurrent state updates with input-dependent parameters while maintaining linear complexity with respect to sequence length. This recurrent architecture

This work is partially funded by the Swedish Research Council, and ELLIIT (Excellence Center at Linköping-Lund in Information Technology), and partially supported by the Competence Center NextG2Com funded by the VINNOVA program for Advanced Digitalisation with grant number 2023-00541. *Corresponding author: FILL THIS.*

The authors are with Department of Electrical and Information Technology, Lund University, Sweden (email:`name.surname@eit.lth.se`).

†Shengjie Chen and †Chenxi Zhang contributed equally to this work.

The complete implementation open source, including the algorithm, fixed-point software reference, and FPGA accelerator code at GitHub [1].

enables it to explicitly explore the temporal information, which is attractive for indoor localization where current position can be predicted using previous positions. Recent studies in other learning-based applications, such as anomaly detection, have also started to explore Mamba-based architectures alongside established CNN- and Transformer-based approaches [?]. However, the use of Mamba for radio-based localization remains relatively unexplored and systematic comparisons of CNN-, Transformer-, and Mamba-based architectures for localization are still lacking.

To make Mamba architecture better suited for our radio-based indoor localization task and efficiently implement on hardware, we simplified and optimized the original mamba algorithm. We also compared the proposed model with existing FCNN-, CNN-, and attention-based localization methods to evaluate its performance against different learning-based architectures. The final accelerator architecture is named as Slim-Mamba. FPGA acceleration is chosen for its reconfigurability, allowing rapid algorithm refinement and hardware prototyping.

For hardware optimization, existing Mamba accelerators have explored computation reordering, accurate quantization and pipelined execution dataflow, and hybrid dataflows [15]–[18]. These works suggest that the efficiency of Mamba acceleration depends on reusable hardware structures that maximize resource sharing, efficient pipelined dataflows that improve processing throughput and reduce inter-stage stalls, and quantization strategies that balance computational efficiency with inference accuracy. Motivated by this observation, this paper explored hardware realization of Slim-Mamba for Massive MIMO indoor localization. The proposed reconfigurable accelerator reuses computation resources across different projection layers, supports configurable execution on Slim-Mamba block level, applies a unified requantization module, and uses fine-grained FIFO-based pipelining to reduce waiting time between projection, nonlinear activation, state update, and output gating.

The main contributions of this paper are summarized as follows:

- A new Slim-Mamba algorithm, a hardware-friendly Mamba-style model for Massive MIMO indoor localization. The model simplifies the original Mamba structure with a small size while preserving centimeter-level accuracy.
- A reconfigurable Slim-Mamba accelerator for FPGA implementation. The architecture reuses shared MAC Array across global projection layers, supports configurable inter-block execution, quantization module, and uses fine-grained FIFO-based pipelining in local layers to reduce stalls, so that the accelerator achieves both efficient resource utilization and low inference delay.
- Real-world indoor Massive MIMO measurements are used instead of relying only on simulated inputs and the performance of Slim-Mamba is compared with FCNNs, CNNs and Attention based localization methods.

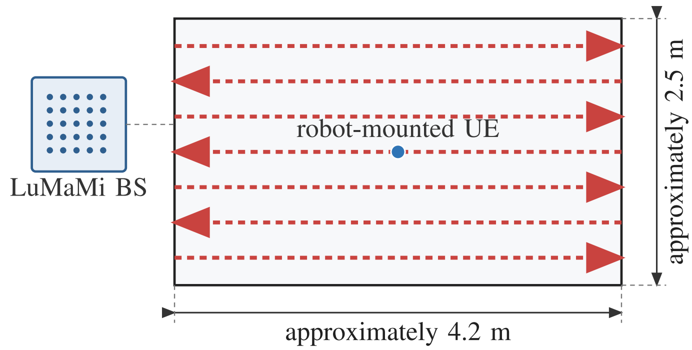


Fig. 1. Schematic of the grid trajectories in the LuViRA dataset evaluated in this work, redrawn from the measurement description in [2], [10].

II. Mamba-Based Model Exploration for Localization

A. Localization Task and Sequence Input

The target task is radio-based indoor localization on the LuViRA dataset, where each sample is represented by a short sequence of Massive MIMO channel observations. In the deployed setting, each observation contains 2100 input features derived from the CIR and power representation used consistently in training, software evaluation, and hardware inference. For one inference window, the model receives $\mathbf{X} = [\mathbf{x}_{t-K+1}, \ldots, \mathbf{x}_t]$, where $\mathbf{x}_i \in \mathbb{R}^{D_{\text{in}}}$ and $D_{\text{in}} = 2100$, and predicts the two-dimensional user position. In this task, the useful temporal information is concentrated in short-range channel variation rather than in long-range sequence dependency.

Figure 1 summarizes the measurement geometry of the radio grid trajectories evaluated in this work. The robot-mounted UE moves along a dense raster of approximately parallel straight paths within the measurement area, with alternating traversal directions and about 0.05 m spacing between adjacent paths. This geometry provides smooth local motion and dense spatial coverage, both of which can favor short-window sequence models.

This property guides the model exploration. The original Mamba block is attractive because it combines recurrent state propagation with input-dependent modulation, but its full selective state-space parameterization was introduced for more general sequence modeling problems. For the present localization setting, the main requirement is to retain short-window temporal tracking while keeping the operator set regular for fixed-point FPGA mapping. Accordingly, the model exploration in this work seeks the smallest Mamba-like recurrent structure that preserves the localization accuracy required for deployment.

B. From Original Mamba to Slim-Mamba

The original Mamba block in [14] combines a local convolutional front end with a selective state-space recurrence. After RMSNorm and input projection, the input is split into a state-update path and a gate path. The state-update path generates input-dependent parameters, performs continuous-to-discrete conversion, and applies the selective scan, while the gate path

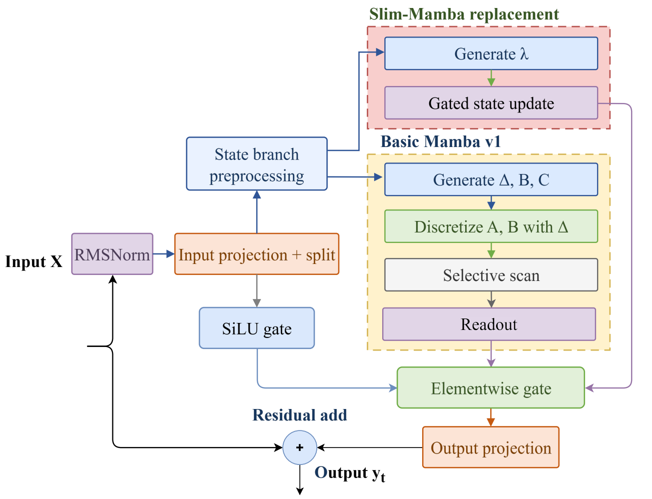


Fig. 2. Structural comparison between the original Mamba block and the proposed Slim-Mamba block.

TABLE I
DEPLOYED SLIM-MAMBA CONFIGURATION AND PARAMETER ROLES.

Symbol	Value	Role in the deployed model
D_{in}	2100	Input feature dimension for each radio observation.
K	2	Temporal window length, i.e., the number of consecutive observations processed in one inference window.
proj_dim	64	Feature dimension after the input projection and before the final regression head.
d_{model}	128	Internal hidden dimension used inside each Slim-Mamba block.
n_{layer}	4	Number of stacked Slim-Mamba blocks in the deployed model.
P	2	Patch length used in the Conv1d patch-embedding stage.
S	2	Patch stride used in the Conv1d patch-embedding stage; $P = S = 2$ yields non-overlapping patches.

modulates the state output before the final output projection and residual addition. This design is effective for general sequence modeling, but it also introduces a more complex operator set for fixed-point FPGA mapping.

The exploration considered two reference directions. The first was a Vanilla Mamba v1 implementation following the original selective state-space formulation in [14]. The second directly adopted the original CMamba architecture in [19], including its feature-fusion branches for cross-channel interaction modeling, and retrained it for LuViRA dataset with task-specific input and output dimensions. CMamba achieved the best software-only accuracy when augmented input features were available, but at the cost of substantially larger model size, longer training time, and a more complex hardware mapping. In contrast, the Vanilla Mamba baseline showed that the full selective-scan path was not necessary to capture the short-window localization dynamics in the LuViRA dataset. These observations motivated the final Slim-Mamba design.

Slim-Mamba follows a localization-driven and deployment-oriented simplification principle. It retains the overall block structure of the original Mamba block, including RMSNorm, input projection and split, element-wise output gating, output projection, and residual addition. As summarized in Fig. 2, the main modification is confined to the state-update path. In the original Mamba block, that path generates input-dependent Δ_t, B_t, and C_t, performs continuous-to-discrete conversion, and then applies the selective scan recurrence. Slim-Mamba removes these selective SSM operators and replaces them with a direct discrete-time gated update. More specifically, after $\mathbf{x}_t$ passes through RMSNorm and the input projection-and-split stage, the state-update path applies depthwise convolution and SiLU to produce the state-update-path feature $\tilde{\mathbf{x}}_t^{(s)}$. Unlike the gate path, this branch does not generate an output gating signal. Instead, it produces the recurrent update coefficient λ_t. For clarity, this branch is referred to as the update-coefficient projection throughout this paper. The single input-dependent update coefficient is then generated as

$$\lambda_t = \sigma\left(W_\lambda * \tilde{\mathbf{x}}_t^{(s)} + b_\lambda\right), \tag{1}$$

where $\sigma(\cdot)$ denotes the sigmoid function. Instead of discretizing a continuous-time state-space model, Slim-Mamba updates the recurrent state directly as

$$\mathbf{h}_t = \lambda_t \odot \mathbf{h}_{t-1} + (1 - \lambda_t) \odot \tilde{\mathbf{x}}_t^{(s)}. \tag{2}$$

The updated state is then modulated by the gate path

$$\mathbf{g}_t = \text{SiLU}(\mathbf{x}_t^{(g)}), \tag{3}$$

and mapped to the block output through

$$\mathbf{y}_t = W_{\text{out}}(\mathbf{h}_t \odot \mathbf{g}_t). \tag{4}$$

Compared with the original Mamba block, the replacement in Fig. 2 removes three major sources of hardware complexity: time-varying generation of multiple SSM parameters, the discretization step from Δ_t to $(\bar{A}_t, \bar{B}_t)$, and the explicit (C_t, D)-based readout. The resulting model remains recurrent and input-dependent, but its operator set is reduced to projection, element-wise nonlinearities, gated state update, and output modulation. This reduced operator set is the main reason why Slim-Mamba is better suited to shared MAC reuse, fixed-point quantization, and fine-grained pipelining.

C. Deployed Configuration

The deployed Slim-Mamba configuration is summarized in Table I. The selected setting preserves the compact short-window behavior observed in software while keeping the projection and state-update dimensions compatible with the shared internal vector widths and buffering structure of the target FPGA accelerator.

The trainable parameter breakdown is summarized in Table II. This parameter distribution is also relevant from the hardware perspective. Most trainable weights are concentrated in the repeated input projection, update-coefficient projection, and output projection paths. These are exactly the stages later

TABLE II
TRAINABLE PARAMETER BREAKDOWN OF THE SLIM-MAMBA CONFIGURATION.

Component	Parameters
Input projection $D_{\text{in}} \rightarrow$ proj_dim	134,464
Patch embedding Conv1d	16,512
Four block RMSNorm layers	512
Four block input-projection layers	262,144
Four block update-coefficient projection layers	263,168
Four block output-projection layers	131,072
Final RMSNorm	128
Flat output layer	8,256
Regression head proj_dim $\rightarrow$ 2	130
Total	**816,386**

mapped to the shared reconfigurable MAC array. Therefore, the algorithmic simplification and the hardware architecture are aligned not only at the level of recurrence definition, but also at the level of parameter concentration and dataflow structure.

III. RECONFIGURABLE SLIM-MAMBA HARDWARE DESIGN AND PIPELINING

This section presents the reconfigurable hardware architecture of the proposed Slim-Mamba accelerator. The proposed architecture supports multi-level reconfigurability from the top-level execution mode to the low-level datapath configuration, improving hardware utilization and enabling flexible adaptation to different computational and numerical requirements. Such multi-level control improves hardware utilization and resource efficiency. Specifically, the term "reconfigurable" refers to:

- **R1:** Configurable inter-block execution dataflow.
- **R2:** Reconfigurable projection scheduling with reusable MAC arrays.
- **R3:** Configurable fixed-point requantization, as illustrated in Fig. 7.

A. Board-Level Dataflow and Four-Block Execution

Fig. 3 shows the overall architecture. The PL-side shell connects the AXI interfaces to the four-block Slim-Mamba compute chain, as shown in Fig. 3(a). The processor-side control logic initializes the accelerator and loads the AXI-Stream data into the buffer of Block 0. After the preload stage is complete, the four-block controller starts the block computation sequence. Each block implements one full Slim-Mamba block. Between adjacent blocks, the current block output is combined with the residual path using saturated fixed-point addition and then written into the next block input buffer. The output of the last block is streamed to the DMA output path.

R1 appears at the block boundary. The four-block chain supports two inter-block transfer modes through the inter-block pipeline control. When the pipeline is enabled, the accelerator uses a streaming inter-block transfer mode. In this mode, the output of Block (N) does not need to wait until the full block result has been cached. Instead, a registered inter-block write channel forwards the block output row by row or tile by tile into the input buffer of Block (N+1). Therefore, partial outputs from the current block can be made available to the next block earlier, improving the opportunity for overlap between block output generation and next-block preparation.

When the pipeline is disabled, the accelerator uses a conservative preload mode. The output of the current block is first accumulated or cached completely. It is then preloaded into the input buffer of the next block, and the next block starts only after this preload step is complete. This mode has simpler control behavior and is useful for validation and debugging, while the pipeline mode is more throughput-oriented. Both modes use the same four-block hardware chain. The difference is the scheduling policy and the inter-block write behavior. This dual-mode configuration allows both debugging convenience and high-performance operation. The preload mode has a simpler control flow, which makes it easier to verify correctness and modify internal logic during development. The pipeline mode, on the other hand, achieves higher throughput and is intended for final deployment.

B. Reconfigurable MAC Array

Inside each Slim-Mamba block, the main MAC-intensive operators are the input projection, the update-coefficient projection, and the output projection. Although these operators use different input buffers, weight banks, and output destinations, they can all be represented as matrix-vector multiplication. The accelerator therefore avoids instantiating independent MAC arrays for these projection layers. Instead, it uses one shared $4 \times 4 \times 4$ MAC array, as illustrated in Fig. 3(b).

Although all projection layers perform matrix-vector multiplication and use the same MAC array to do the calculation, their varying dimensions require dedicated schedulers for data tiling. The input-projection scheduler reads the block input or hidden representation and the input-projection weights, and converts them into activation and weight tiles. The update-coefficient scheduler reads the Slim-Mamba state-update input cache and the update-coefficient projection weights, and generates the corresponding tile stream. The output-projection scheduler reads the gated state-update output and the output-projection weights. Although these three schedulers access different buffers and write to different destinations, they expose the same fabric interface. A MAC array manager arbitrates among the schedulers and configures the shared fabric to serve one projection role at a time.

The shared scheduling mechanism described above implements **R2**, allowing the same MAC array to be reused across the projection operators of a Slim-Mamba block. This reduces DSP duplication and avoiding the placement overhead of separate arrays. The fabric interface includes control parameters that determine the computation dimensions, allowing the $4 \times 4 \times 4$ array to support different logical matrix shapes, reduction lengths, and output forms.

The $4 \times 4 \times 4$ fabric contains four 4×4 sub-arrays. Each sub-array receives a four-element activation slice and a 4×4 weight block. The four sub-arrays operate with a staggered

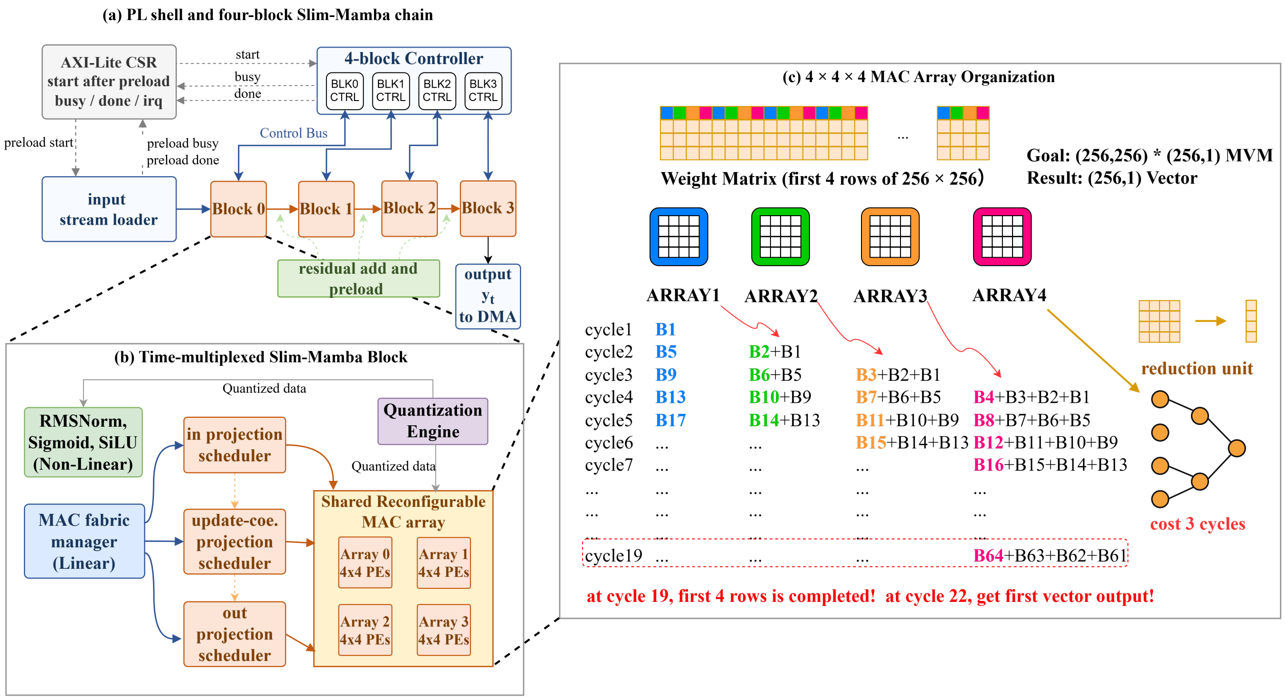


Fig. 3. Reconfigurable Slim-Mamba accelerator architecture: (a) PL shell and four-block Slim-Mamba chain, (b) time-multiplexed Mamba block with shared projection fabric, and (c) $4 \times 4 \times 4$ MAC Array Organization.

schedule and their partial results are combined through a reduction tree. Fig. 3(c) illustrates the array organization for the update-coefficient projection in the state-update path. The symbol B in the table denotes the current 4×4 input tile. The weight matrix shown below represents a portion of the update-coefficient weight matrix, where each square corresponds to one 4×4 block. The four sub-arrays operate in a staggered pipeline. At cycle 19, the first four rows of the weight matrix have been fully consumed, and at cycle 22 the first output tile is produced after the reduction tree.

The PE-array organization should be matched to the data-supply and reduction pattern of the Slim-Mamba workload. Systolic arrays are a classical and widely used organization for regular matrix computations [20], [21], while previous researches have shown that data movement, local reuse, and dataflow mapping are critical to energy efficiency and throughput [22], [23]. Because the main computation in Slim-Mamba projection operators is matrix-vector multiplication, compared with systolic array, our $4 \times 4 \times 4$ arrays organization reduces global routing and data-supply pressure for the MVM-dominated workload. Compared with a 64×1 GEMV engine, which is designed for MVM calculations, it consumes the same amount of hardware resources but provides earlier tile availability. Still take Slim-Mamba state update datapath as example, which performs MVM with the matrix shape of (256,256) and vector shape of (256,1). With the four 4×4 sub-arrays working in pipeline, the first output vector can be produced after 22 cycles rather than waiting for a full vector-reduction interval in a 64×1 GEMV engine, which is 256 cycles. This earlier tile output is important because the following nonlinear and state-update stages can start before the whole projection vector is completed.

C. *Fine-Grained FIFO-Based Pipeline*

As discussed above, the PEs inside the $4 \times 4 \times 4$ MAC array can be configured for MVM, EWM, and EWA, so the tradeoff between hardware reuse and latency must be considered. A coarse implementation of the state-update path would execute update-coefficient projection, sigmoid, state update, SiLU, and output gating sequentially. This maximizes resource reuse, but it forces each stage to wait until the previous stage has produced a full vector. The proposed accelerator instead uses a fine-grained FIFO-based pipeline. Once the MAC array produces and reduces an output tile, the tile is forwarded through FIFO interfaces to the bias-add, sigmoid, and state-update stages. The state-update unit starts as soon as the corresponding tile, update coefficient, state value, and scale are available. It does not wait for the full projection vector.

The state update follows the Slim-Mamba recurrence in Eq. (2), where λ_t is generated by the update-coefficient projection followed by the sigmoid unit. The updated state is then element-wise multiplied by the SiLU gate output. FIFOs are used to align the projection output, sigmoid value, gate path, state read result, and dynamic scale. This alignment is necessary because different sub-stages have different local latencies.

For an N-tile Slim-Mamba state-update-path workload, the coarse-grained schedule can be abstracted as

$$T_{\text{coarse}} = 29N, \tag{5}$$

where projection, nonlinear activation, state update, and output gating are mostly serialized. With fine-grained FIFO streaming, the schedule becomes

$$T_{\text{fine}} = 22N + 7. \tag{6}$$

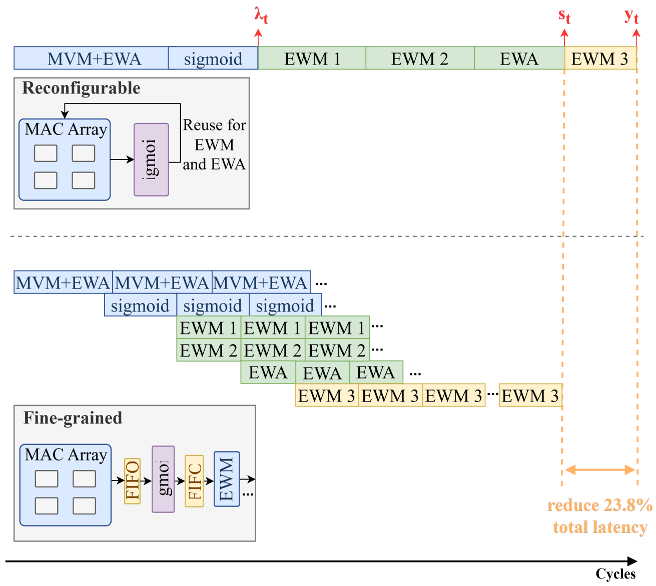


Fig. 4. Comparison between coarse-grained and fine-grained scheduling.

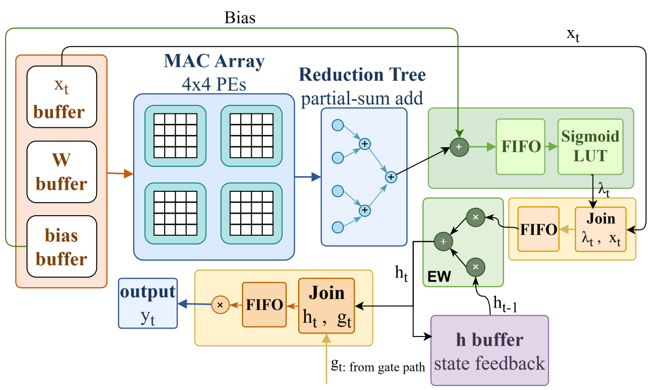


Fig. 5. Slim-Mamba state-update datapath.

For $N = 64$, this reduces the state-update-path latency from 1856 cycles to 1415 cycles, corresponding to a 23.8% reduction. The additional cost is about 12 DSPs per state-update path, which is acceptable compared with the latency reduction. Figure 4 shows the comparison. Fig. 5 shows the final fine-grained Slim-Mamba state-update datapath, where the hardware modules implementing the operations in Eqs. (1)–(4), including recurrent state update, bias addition, sigmoid, SiLU gating, and output generation, are connected through FIFO-based interfaces.

D. Quantization Strategy and Hardware implementation

As the quantization strategy depends on the data distribution, we first profile the floating-point activation data range of the trained Slim-Mamba model using 4096 representative input samples. The accelerator processes activations as 4-element vectors, where each vector contains four adjacent channel values that are handled in parallel by the datapath. Therefore, the activation range is profiled at the same 4-lane vector granularity.

TABLE III
STAGE-LEVEL QUANTIZATION ERROR WITH DYNAMIC SCALING APPLIED TO THE STATE PATH.

Stage	Fixed-scale MAE	Dynamic-scale MAE
RMSNorm	0.00	0.00
Input proj. (state path)	0.01	0.01
Input proj. (gate path)	0.01	0.01
Update coefficient λ_t	0.01	0.01
SiLU gate	0.01	0.01
coefficient projection	0.01	0.01
Gate-path sigmoid	0.00	0.00
State update	**0.13**	**0.09**
Element-wise gating	0.03	0.03
Output projection	0.03	0.03

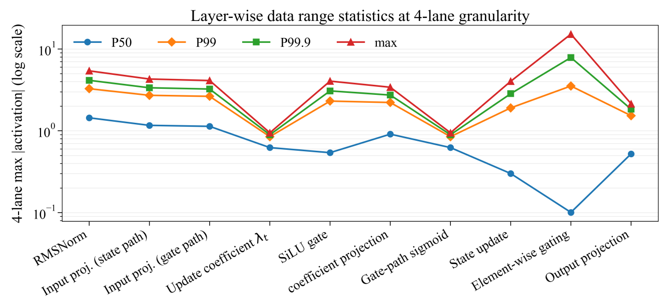


Fig. 6. Data range statistics

The profiling result in Fig.6 shows that the required numerical range is strongly layer-dependent. The update coefficient and sigmoid layers have a compact range, while the recurrent state and element-wise gating layers show a much larger gap between the median and tail values. In particular, the state-update path and corresponding gating path has a small median range but significantly larger P99 and maximum values, indicating a long-tailed distribution across channel groups and input samples. Therefore, when determining the scaling strategy, the state-update layer requires a dedicated quantization configuration to accommodate its dynamic data range. The strategy is called dynamic scaling, which means the scale factor is computed from the current recurrent-state range and used when storing and restoring the state. This allows the state representation to adapt to data ranges that vary over time.

Table III shows the experimental results for scaling strategy. We first evaluate a fixed-scale baseline because it avoids runtime dynamic scale generation, per-layer scale storage, and scale-alignment control. In this baseline, fixed-point data are represented in Q8.8, where the fixed scale is $s = 2^{-8} = 1/256$ and $x_{\text{real}} = s \cdot x_{\text{int}}$. Q8.8 is used as the fixed-scale baseline because it offers a balanced integer/fractional split. The fixed-scale baseline produces small errors remain at 0.00–0.01 MAE for all layers except the state-update path. In contrast, the state-update MAE reaches 0.13, which validates the profiling result in Fig. 6. After dynamic scaling is applied to the state update path, the MAE is reduce to 0.09.

After identifying the scaling strategy, the exact Q format, rounding mode, and saturation mode for each layer are selected through statistics-guided simulation search. The software search first inserts quantize–dequantize operations

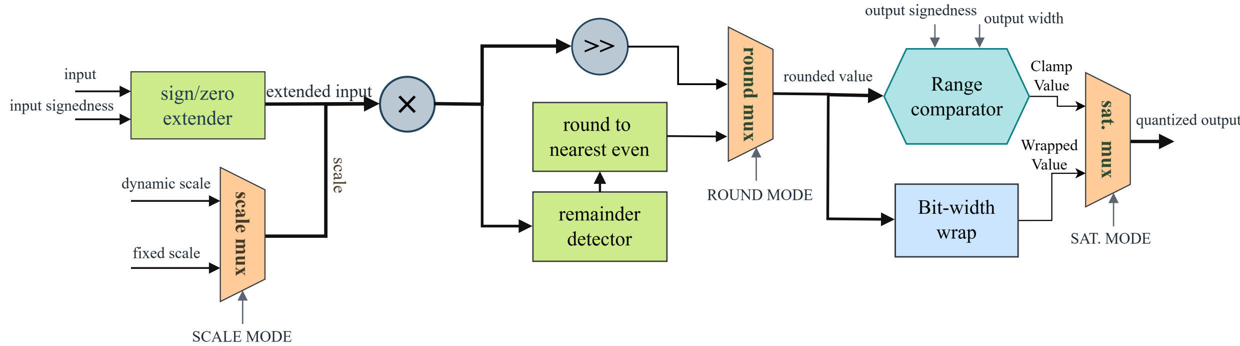


Fig. 7. Configurable fixed-point requantization engine reused across projection, state-update, and residual paths.

at layer boundaries to estimate the numerical loss of each candidate configuration [24], [25]. The promising candidates are then verified using a bit-true C++ model that follows the RTL behavior, including binary-point shifts, round-to-nearest-even, saturation, and runtime state-scale conversion.

The quantization strategy is implemented by a shared reconfigurable requantization engine rather than by instantiating a separate quantization unit for every layer. This module is used after MAC accumulation, Q-format conversion, state update, scale multiplication, and residual addition. As shown in Fig. 7, this shared structure reduces duplicated hardware and allows the layer-wise quantization policy obtained from software analysis to be adjusted easily to hardware control parameters.

For the dynamic-scale state-update path, it requires additional alignment logic. Since the state-update datapath is stream based, each token, state address, update coefficient, and runtime scale must arrive at the state-update unit in the same order. A ping-pong scale buffer is used for this purpose, as seen in Fig. 8. While one bank receives the incoming tokens, their state addresses, and the generated runtime scales, the other bank provides an already aligned token-scale pair to the downstream state-update pipeline. The write-buffer selector and read-buffer selector identify the active bank. The read and write banks switch between consecutive tokens, which hides the scale-generation latency and preserves streaming order. In this way, dynamic scaling can be inserted into the state-update path without breaking the fine-grained FIFO-based pipeline.

The final fixed-point model achieves an output MAE of 0.02 with respect to the floating-point output and a mean localization error of 0.14 m, which is close to the float-point reference, showing that the selected layer-wise fixed-point strategy preserves the localization accuracy while remaining compatible with the hardware datapath.

E. Nonlinear Layer Implementation

From the algorithm analysis in Section II, the main nonlinear operators in the Slim-Mamba model are RMSNorm, sigmoid, and SiLU.

RMSNorm is placed before the projection path to stabilize the dynamic range of the input features. For an input vector

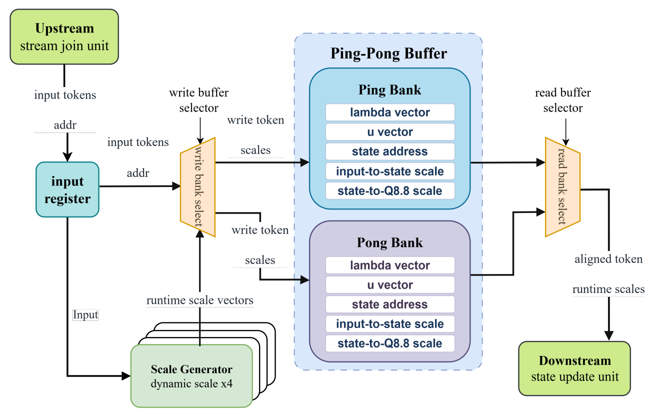


Fig. 8. Ping-pong scale-buffer organization for aligning token, state address, update coefficient, and runtime scale streams.

$\mathbf{x} = [x_0, x_1, \ldots, x_{D-1}]$, the RMS value is

$$\mathrm{RMS}(\mathbf{x}) = \sqrt{\frac{1}{D}\sum_{i=0}^{D-1} x_i^2 + \epsilon}, \tag{7}$$

and the normalized output is computed as

$$\hat{x}_i = x_i \cdot \widehat{\frac{1}{\mathrm{RMS}(\mathbf{x})}}. \tag{8}$$

The square-root operation is implemented with a trial-quotient digit-recurrence procedure, which uses shift, comparison, and subtraction operations instead of a general floating-point square-root unit. This design follows the general principle of digit-recurrence square-root methods, where the root is formed iteratively through residual comparison and root-digit selection [26], [27]. The final division is replaced by reciprocal generation followed by multiplication. A Q30 reciprocal is used so that the shared normalization scale remains accurate for all channels of one token.

The sigmoid function is used to generate the state-update coefficient:

$$\sigma(x) = \frac{1}{1+e^{-x}}. \tag{9}$$

Direct evaluation of the exponential function is costly in fixed-point digital hardware. Therefore, lookup-table-based approximations have been widely used for hardware implementations

TABLE IV
SOFTWARE-SIDE MODEL COMPARISON.

Model	Input features	Model size (MB)	Avg. training time / epoch (s)	Train rate (batch/s)	Mean localization error (cm)	Deployment complexity
FCNN [10]	CIR	789	N/A	N/A	~ 13	High
CMamba [19]	CIR + Power + amplitude	40.4	73	13.18	12.9	High
Vanilla Mamba v1	CIR + Power	3.38	33	32.62	24.74	Medium
Slim-Mamba	CIR + Power	3.40	13	75.30	13.5	Low

of sigmoid and other neural-network activation functions [28], [29]. In this work, the sigmoid input is clamped to $[-4, +4)$, and a 2048-entry LUT stores the corresponding Q0.16 sigmoid values. This avoids evaluating the exponential function in hardware. SiLU reuses the same sigmoid path:

$$\mathrm{SiLU}(x) = x \cdot \sigma(x). \tag{10}$$

The original input is delayed through a FIFO while the sigmoid value is produced. After alignment, an element-wise multiplier generates the SiLU output. Therefore, sigmoid and SiLU share the clamp and LUT logic, while SiLU only adds FIFO alignment and vector multiplication.

IV. RESULTS

A. Model Selection

The software exploration was used to identify a localization model that is not only accurate on LuViRA but also practical to quantize and map to a reconfigurable accelerator. Table IV summarizes the main model variants considered in this study. The FCNN baseline provides a strong accuracy reference, but at a very large model size. Vanilla Mamba v1 is smaller, yet its localization error is substantially worse on this task. The original CMamba [19], retrained for LuViRA localization, reaches the best software-only accuracy when additional amplitude features are used, but this comes with a larger model, longer training time, and a more complex multi-branch hardware mapping.

The Mamba-family results in Table IV were produced with the LuViRA radio split used in this work: even-indexed trajectories form the held-out test partition, odd-indexed trajectories form the training pool, and two trajectories are randomly selected from that odd-indexed pool for validation. The FCNN entry is the published reference result from [10] and is included only as an accuracy and model-size reference. The Mamba-family models were trained in PyTorch on an Intel Core i5-13600KF host with an NVIDIA RTX 3080 GPU using AdamW, a batch size of 32, an initial learning rate of 3×10^{-4}, cosine learning-rate decay, and seed 42. In Table IV, "Avg. training time / epoch" denotes the measured elapsed time required to complete one training epoch on the same hardware platform, "Train rate" denotes the corresponding mini-batch processing rate, "Mean localization error" denotes the average Euclidean distance between the predicted and ground-truth 2-D positions over the test set, and "Deployment complexity" is a qualitative summary of the expected hardware-mapping difficulty based on operator diversity, branching structure, and recurrent-state handling.

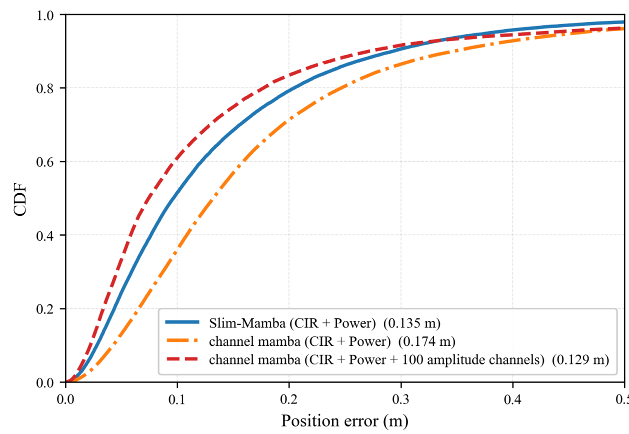


Fig. 9. Cumulative distribution of localization error for Slim-Mamba and two CMamba reference runs under matched-input and augmented-input settings.

Because the odd/even split interleaves nearby parallel trajectories, these results primarily measure interpolation within the densely sampled raster-scan scenario. They should not be interpreted as evidence of generalization to arbitrary curved trajectories or substantially different motion patterns.

Figure 9 isolates the effect of the input representation. Here, "matched inputs" means that Slim-Mamba and CMamba both use the same CIR+Power input representation, whereas "augmented inputs" refers to the additional amplitude channels used by the strongest CMamba configuration. Under matched inputs, Slim-Mamba delivers the better error distribution over most of the operating range. The augmented-input CMamba remains the strongest software-only accuracy reference, but this comes at the cost of a much larger model and a more complex multi-branch mapping. The Slim-Mamba algorithm was therefore selected for the subsequent hardware implementation study.

The temporal window length was also studied because it affects both algorithmic context and hardware buffering cost. Table V shows that preferred deployment point lies in the short-window regime.

B. Throughput, Resource Utilization, and Power

The final fixed-point design preserves the software-side localization accuracy: the hardware fixed-point model reaches an output MAE of 0.02 with respect to the floating-point reference and a mean localization error of 0.14 m, which matches the software fake-quantization reference. At 100 MHz, the accelerator reaches 471 positions/s, or about

TABLE V

REPRESENTATIVE TEMPORAL-WINDOW ABLATION FOR SLIM-MAMBA.

K (patch, stride)	Eval err. (cm)	Test err. (cm)
1 (1,1)	10.07	**13.50**
2 (2,2)	9.56	**13.50**
8 (2,2)	9.41	14.10
4 (4,4)	9.98	13.70
8 (4,4)	9.75	14.40
8 (8,4)	9.74	15.20
32 (8,4)	9.33	16.60

TABLE VI

POST-SYNTHESIS RESOURCE UTILIZATION AT 100 MHZ.

Resource	Used	Available	Utilization
LUT	60,868	274,080	22.21%
FF	77,345	548,160	14.11%
BRAM tiles	279	912	30.59%
DSP	496	2,520	19.68%

TABLE VII

ON-CHIP POWER ESTIMATE.

Metric	Value	Share
Total on-chip power	4.29 W	100%
Dynamic power	3.56 W	83%
Static power	0.73 W	17%
PS dynamic power	2.62 W	74% of dynamic power
PL dynamic power	0.94 W	26% of dynamic power

$4.7\times$ the 100 positions/s real-time target. This corresponds to an average processing time of 2.12 ms per position estimate.

Table VI shows that the shared MAC array and time-multiplexed block structure keep the implementation within the device budget while preserving the full end-to-end deployment flow on the target FPGA.

Table VII indicates that platform-side data movement and processing-system activity remain the dominant contributors to on-chip power. This suggests that further system-level optimization should focus on memory traffic and host-platform overhead in addition to arithmetic efficiency.

To place these implementation results in context, Table VIII summarizes representative massive-MIMO localization baselines spanning a software-only FCNN, a CNN-oriented accelerator, a programmable floating-point ASIP platform, and a Transformer-based hardware design. The table reports the dataset/scenario, implementation platform, clock frequency, arithmetic format, throughput, and localization accuracy as stated in the cited works. When a field is not explicitly reported in the source, it is marked as N/A.

C. Latency Reduction from Fine-Grained Pipelining

The final set of results concerns the FIFO-based fine-grained pipeline introduced in the state-update path. As analyzed in Section III, the coarse schedule can be approximated by $T_{\text{coarse}} = 29N$, whereas the fine-grained schedule becomes $T_{\text{fine}} = 22N + 7$. For $N = 64$ tiles, this reduces the Slim-Mamba state-update path latency from 1856 cycles to 1415 cycles, corresponding to a 23.8% reduction.

The per-block latency breakdown is reported in Table IX. One Slim-Mamba block requires 5301 cycles, or 53.01 μs at 100 MHz. Input projection remains the dominant stage, but the latency-sensitive state-update path is no longer the main bottleneck. This is the intended effect of the fine-grained pipeline: the recurrence and nonlinear stages start on partial tiles instead of waiting for a full projection vector.

V. CONCLUSIONS

This paper presented Slim-Mamba, a deployment-oriented Mamba-based localization model and its reconfigurable FPGA accelerator for Massive MIMO indoor localization. The main contribution is the combination of three elements: a task-driven simplification of the original Mamba state-update path, a layer-wise fixed-point strategy with dynamic scaling for the recurrent state update, and a hardware architecture that maps the resulting operator set onto a shared reconfigurable datapath. Relative to the original Mamba formulation, Slim-Mamba retains recurrent temporal modeling and input-dependent gating while removing operators that are less suitable for efficient FPGA deployment. Using real indoor localization measurements from LuViRA, the final hardware-oriented model achieves a mean localization error of 0.14 m, and the accelerator reaches 471 positions/s at 100 MHz with moderate FPGA resource usage. These results show that Slim-Mamba provides an effective bridge between sequential localization models and practical low-latency hardware deployment.

REFERENCES

[1] Elison-debug, "Slim mamba accelerator fpga implementation," https://github.com/Elison-debug/slim_mamba, 2025.

[2] I. Yaman, G. Tian, M. Larsson, P. Persson, M. Sandra, A. D"urr, E. Tegler, N. Challa, H. Garde, F. Tufvesson, K. Åstr"om, O. Edfors, S. Malkowsky, and L. Liu, "The LuViRA dataset: Synchronized vision, radio, and audio sensors for indoor localization," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 11 920–11 926.

[3] S. E. Trevlakis, A.-A. A. Boulogeorgos, D. Pliatsios, J. Querol, K. Ntontin, P. Sarigiannidis, S. Chatzinotas, and M. Di Renzo, "Localization as a key enabler of 6g wireless systems: A comprehensive survey and an outlook," *IEEE Open Journal of the Communications Society*, vol. 4, pp. 2733–2801, 2023.

[4] C. Xiang, Z. Zhang, S. Zhang, S. Xu, S. Cao, and V. Lau, "Robust submeter level indoor localization - a logistic regression approach," in *ICC 2019 - 2019 IEEE International Conference on Communications (ICC)*, 2019, pp. 1–6.

[5] F. Shi, W. Li, C. Tang, Y. Fang, P. V. Brennan, and K. Chetty, "Decimeter-level indoor localization using wifi round-trip phase and factor graph optimization," *IEEE Journal on Selected Areas in Communications*, vol. 42, no. 1, pp. 177–191, 2024.

[6] F. Zafari, A. Gkelias, and K. K. Leung, "A survey of indoor localization systems and technologies," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 3, pp. 2568–2599, 2019.

[7] I. Yaman, G. Tian, E. Tegler, J. Gulin, N. Challa, F. Tufvesson, O. Edfors, K. Åstr"om, S. Malkowsky, and L. Liu, "LuViRA dataset validation and discussion: Comparing vision, radio, and audio sensors for indoor localization," *IEEE Journal of Indoor and Seamless Positioning and Navigation*, vol. 2, pp. 240–250, 2024.

[8] S. Malkowsky, J. Vieira, L. Liu, P. Harris, K. Nieman, N. Kundargi, I. C. Wong, F. Tufvesson, V. "Owall, and O. Edfors, "The world's first real-time testbed for massive MIMO: Design, implementation, and validation," *IEEE Access*, vol. 5, pp. 9073–9088, 2017.

TABLE VIII
CONTEXTUAL COMPARISON WITH REPRESENTATIVE LOCALIZATION MODELS AND ACCELERATORS.

Work	**Backbone**	**Dataset / scenario**	**Platform**	**Clock**	**Arith.**	**Real data**	**Throughput**	**Localization accuracy**
Tian *et al.* [10]	Dual FCNN	Indoor 3.7 GHz massive MIMO testbed	SW only	N/A	Float	Yes	N/A	centimeter-level accuracy
Attari *et al.* [11]	CNN	Massive-MIMO positioning	GF-22 nm vector processor	555 MHz	N/A	Yes	271 positions/s	40 cm average error
Attari *et al.* [12]	CNN + ASIP	Massive-MIMO comm. + positioning	22 nm FD-SOI ASIP + accelerators	800 MHz	Float	Yes	$\sim$ 390 positions/s	N/A
Yaman *et al.* [13]	Adaptive Transf.	Outdoor massive MIMO	Xilinx Zynq UltraScale+ FPGA	100 MHz	16B	Yes	up to 1961 positions/s	< 1.15 m
This work	Slim-Mamba	Indoor LuViRA	Xilinx Zynq UltraScale+ FPGA	100 MHz	FP16	Yes	471 positions/s	0.14 m mean error

TABLE IX
LATENCY BREAKDOWN PER SLIM-MAMBA BLOCK.

Stage	**Cycles**	**Time at 100 MHz**
Input projection	2,292	22.92 μs
State-update path	1,415	14.15 μs
Output projection	1,093	10.93 μs
Control / alignment overhead	501	5.01 μs
Total per block	**5,301**	**53.01 μs**

[9] G. Tian, I. Yaman, M. Sandra, X. Cai, L. Liu, and F. Tufvesson, "High-precision machine-learning based indoor localization with massive MIMO system," in *ICC 2023 – IEEE International Conference on Communications*, Rome, Italy, 2023, pp. 3690–3695.

[10] ——, "Deep-learning-based high-precision localization with massive MIMO," *IEEE Transactions on Machine Learning in Communications and Networking*, vol. 2, pp. 19–33, 2024.

[11] M. Attari, J. R. Sánchez, L. Liu, and S. Malkowsky, "An application specific vector processor for cnn-based massive mimo positioning," in *2021 IEEE International Symposium on Circuits and Systems (ISCAS)*, 2021, pp. 1–5.

[12] M. Attari, O. Edfors, and L. Liu, "Accelerator-assisted floating-point asip for communication and positioning in massive mimo systems," *arXiv preprint arXiv:2502.09785*, 2025.

[13] I. Yaman, S. Cheng, O. Edfors, and L. Liu, "Efficient implementation of an adaptive transformer accelerator for massive mimo outdoor localization," *arXiv preprint arXiv:2605.13507*, 2026.

[14] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," *arXiv preprint arXiv:2312.00752*, 2023.

[15] R. Wei, S. Xu, L. Zhong, Z. Yang, Q. Guo, Y. Wang, R. Wang, and M. Li, "LightMamba: Efficient mamba acceleration on FPGA with quantization and hardware co-design," in *2025 Design, Automation & Test in Europe Conference (DATE)*, 2025.

[16] A. Wang, H. Shao, S. Ma, and Z. Wang, "Fastmamba: A high-speed and efficient mamba accelerator on fpga with accurate quantization," in *2025 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, vol. 1, 2025, pp. 1–6.

[17] X. Jin, H. Zheng, M. Nie, J. Wang, and C. R. Shi, "HCSAs: Hybrid computing systolic arrays for accelerating mamba models with unified state space buffers and energy-efficient dataflow," in *Proceedings of the Great Lakes Symposium on VLSI 2025*. New York, NY, USA: ACM, 2025, pp. 457–462.

[18] J. Li, S. Huang, J. Xu, J. Liu, L. Ding, N. Xu, and G. Dai, "Marca: Mamba accelerator with reconfigurable architecture," in *2024 ACM/IEEE International Conference On Computer Aided Design (ICCAD)*, 2024, pp. 1–9.

[19] C. Zeng, Z. Liu, G. Zheng, and L. Kong, "Cmamba: channel correlation enhanced state space models for multivariate time series forecasting," *arXiv preprint arXiv:2406.05316*, 2024.

[20] Kung, "Why systolic architectures?" *Computer*, vol. 15, no. 1, pp. 37–46, 1982.

[21] Z.-G. Liu, P. N. Whatmough, and M. Mattina, "Systolic tensor array: An efficient structured-sparse gemm accelerator for mobile cnn inference," *IEEE Computer Architecture Letters*, vol. 19, no. 1, pp. 34–37, 2020.

[22] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE Journal of Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, 2017.

[23] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," *Proceedings of the IEEE*, vol. 105, no. 12, pp. 2295–2329, 2017.

[24] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2018, pp. 2704–2713.

[25] R. Krishnamoorthi, "Quantizing deep convolutional networks for efficient inference: A whitepaper," *arXiv preprint arXiv:1806.08342*, 2018.

[26] D. Harris, J. Stine, M. Ercegovac, A. Nannarelli, K. Parry, and C. Turek, "Unified digit selection for radix-4 recurrence division and square root," *IEEE Transactions on Computers*, vol. 73, no. 1, pp. 292–300, 2024.

[27] J. D. Bruguera, "Radix-64 floating-point division and square root: Iterative and pipelined units," *IEEE Transactions on Computers*, vol. 72, no. 10, pp. 2990–3001, 2023.

[28] P. Kumar Meher, "An optimized lookup-table for the evaluation of sigmoid function for artificial neural networks," in *2010 18th IEEE/IFIP International Conference on VLSI and System-on-Chip*, 2010, pp. 91–95.

[29] F. Piazza, A. Uncini, and M. Zenobi, "Neural networks with digital lut activation functions," in *Proceedings of 1993 International Conference on Neural Networks (IJCNN-93-Nagoya, Japan)*, vol. 2, 1993, pp. 1401–1404 vol.2.

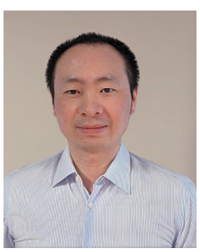
Shengjie Chen received the bachelor's degree in Electronic Science and Technology from Southeast University in 2024 and the master's degree in Embedded Electronics Engineering from Lund University in 2026. Her research interests include machine-learning hardware accelerators and digital integrated circuit design.

Chenxi Zhang received the bachelor's degree from Southern University of Science and Technology and the master's degree in Embedded Electronics Engineering from Lund University. His research interests include machine-learning hardware deployment and acceleration, especially efficient neural processing unit (NPU) architectures and AI inference accelerators.

Ilayda Yaman (Student Member, IEEE) completed her bachelor's degree at Istanbul Technical University in 2018 and her master's degree in Embedded Electronics Engineering at Lund University in 2020. During her master's studies, she received the LU Global Scholarship. She is currently pursuing the Ph.D. degree at Lund University under the supervision of Liang Liu. Her research focuses on low-power machine-learning hardware for radio-based localization systems.

Sijia Cheng (Student Member, IEEE) received a double bachelor's degree from Beijing Jiaotong University and KU Leuven in 2020, followed by the master's degree in Embedded Electronics Engineering from Lund University in 2022. During her studies at Lund, she received the LU Global Scholarship. She is currently working toward the Ph.D. degree at Lund University under the supervision of Liang Liu. Her research focuses on low-latency digital signal processing for massive MIMO systems.

Ove Edfors (Senior Member, IEEE) is a Professor of Radio Systems with the Department of Electrical and Information Technology, Lund University, Sweden. His research interests include statistical signal processing and low-complexity algorithms for wireless communications. In the context of Massive MIMO and large intelligent surfaces, his current research focuses on how realistic propagation characteristics influence system performance and baseband-processing complexity.



Liang Liu (Member, IEEE) received the Ph.D. degree from Fudan University in 2010. He joined Lund University as a postdoctoral researcher. Since 2024, he has been a Professor at Lund University. His research interests include wireless systems and digital integrated circuit design. He is a member of the Technical Committee on VLSI Systems and Applications and the CAS Committee for Communications of the IEEE Circuits and Systems Society.